Learning Task 3 - SNR estimation and enhancement: FIR filters

The purpose of this Learning Task is to let you practice with FIR filter design. You will design and apply frequency selective filters in order to estimate the ratio between the signal and the noise power (SNR) of the signal. We also want to improve the SNR. We will not only design a filter using the frequency sampling method as presented in the lecture. In order to sketch a broader narrative, we also show how you can use Matlab or Python to design a filter using other methods, in particular the window method. Using a tool is good, but you should at least have some notion how it works.

Assume you have a sample $y[n] = x[n] + d[n]$ of length $N = 20000$ consisting of a discrete time signal $x[n]$ polluted with uniform noise $d[n]$. The signal lies somewhere between 1kHz and 2kHz, the uniform noise is equally present at every frequency. The analog signal is filtered before sampling with frequency $f_s = 10kHz$.

Q1 Plot the FFT $|Y[k]|$ of $y[n]$ against k and verify there seems to be uniform noise and four peaks in the signal. Verify that the values of k_1 and k_2 between which the signal is found match what you expect from the given numbers.
H1 Recall that the format of the single frequency ω sample is $x[n] = \cos(\omega Tn) = \cos(\Omega n)$, so $\Omega = \omega T = 2\pi fT = 2\pi\frac{f}{f_s}$. Furthermore, the kth element of $Y[k]$ matches $\Omega_k = \frac{2\pi}{N}k$ or reversely index $k = \frac{\Omega_k N}{2\pi}$ represents frequency Ω_k.
A1 We can use in Matlab

```
...
Y=fft(y);
...
```

or in Python

```
import scipy.fft as scf
...
Y=scf.fft(y);
...
```

If we calculate and plot $|Y[k]|$, we notice that there seems to be a signal between roughly $k_1 = 2000$ and $k_2 = 4000$. The peaks on the right are mirrored values since $y[n]$ is real valued. The observed values for k correspond to (through $k = \frac{\Omega_k}{N}2\pi$):

$$\Omega_{min} = 2\pi\frac{1kHz}{10kHz} = 0.2\pi,\ \Omega_{max} = 2\pi\frac{2kHz}{10kHz} = 0.4\pi,$$

as expected.

Q2 We want to design a bandpass filter that selects the signal between $\Omega_{min} = 0.2\pi$ and $\Omega_{max} = 0.4\pi$ and removes noise as much as possible. We want to use the frequency sampling method. For now, select an odd[1] length filter having K elements. We specify the $\frac{K+1}{2}$ values for $H[k]$

$$H[0],\ H[1], \cdots, H[\frac{K-1}{2}]$$

[1] Even is possible, but would unnecessary complicate matters.

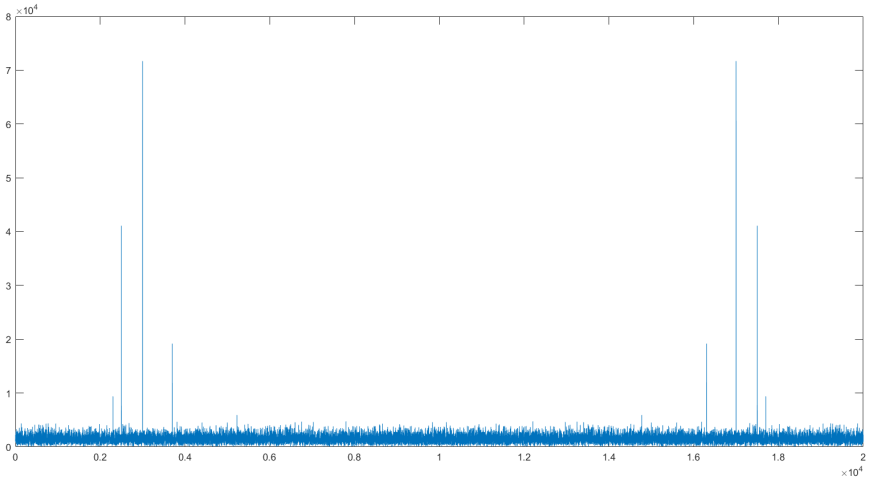


Figure 1: Plot of $|Y[k]|$

which give the desired reponse at frequencies (with $\Omega_0 = \frac{2\pi}{K}$):

$$0 \cdot \Omega_0 = 0,\ 1 \cdot \Omega_0 = \frac{2\pi}{K}, \cdots, \frac{K-1}{2} \cdot \frac{2\pi}{K} = \frac{\pi(K-1)}{K}.$$

To make it more concrete: select $K = 19$, then specify $\frac{19+1}{2} = 10$ values

$$H[0],\ H[1], \cdots, H[9]$$

which give the desired reponse at frequencies (with $\Omega_0 = \frac{2\pi}{19}$):

$$0 \cdot \Omega_0 = 0,\ 1 \cdot \Omega_0 = \frac{2\pi}{19}, \cdots, \frac{K-1}{2} \cdot \frac{2\pi}{19} = \frac{\pi(18)}{19}.$$

Why do we only need to specify slightly more than half of the desired frequency response to get the filter coefficients $h[n]$?
H2 Note that we are actually specifying **absolute** values $|H[k]|$ and **not** $H[k]$ itself. Which freedom do we have and how does that impact $H[k]$ and $h[n]$?
A2 Frequency sampling works by specifying absolute values $|H[k]|$. $H[k]$ is the DFT of $h[n]$ and thus can be complex valued. Any complex number z can be written in a polar form $z = |z|e^{j\arg(z)}$. So, we only specified $|z|$ and there is freedom to pick any value for $\arg(z)$. Without going into too much mathematical detail, a clever choice for $z = H[k]$ is

$$\arg(H[k]) = -\frac{K-1}{2}\frac{2\pi}{K}k = -\frac{(K-1)\pi k}{K}$$

because it guarantees that

- $|H[K-k]| = |H[k]|$, so you only need to specify $|H[0]|,\ |H[1]|, \cdots, |H[\frac{K-1}{2}]|$.
- $h[n]$ has real valued coefficients
- $h[n]$ is a linear phase filter, so we will not encounter phase distortion when we apply the filter

In the case of $K = 19$, this means

$$\arg(H[k]) = -\frac{18\pi}{19}k$$

Also, due to this choice we have

$$\begin{aligned} &|H[0]| \\ &|H[1]| = |H[18]|, \\ &\quad \vdots \quad = \quad \vdots, \\ &|H[9]| = |H[10]| \end{aligned}$$

We conclude that specifying 10 (in general $\frac{K+1}{2}$) values of a filter of length 19 (in general length K) suffices.

Q3 We want to highlight two methods you can use to find the filter coefficients $h[n]$.

Method 1: Calculate $H[k] = |H[k]|e^{j\arg(H(k))} = |H[k]|e^{-j\frac{(K-1)\pi k}{K}}$ for $k = 0, \cdots, \frac{K-1}{2}$ and set $H[N-k] = H[k]^* = |H[k]|e^{j\frac{(K-1)\pi k}{K}}$ for $k = 1, \cdots, \frac{K-1}{2}$. The filter coefficients $h[n]$ are found by taking the inverse DFT of $H[k]$. Note that they are real as we enforced $H[N-k] = H[k]^*$.

Method 2: Due to symmetry in the $H[k]$ values, it is possible to give an explicit formula for $h[n]$:

$$h[n] = \frac{1}{K}\left(|H[0]| + \sum_{k=1}^{\frac{K-1}{2}} 2|H[k]| \cos\left(\frac{2\pi}{K}k\left(n - \frac{K-1}{2}\right)\right)\right), \; n = 0, \cdots, K-1$$

Implement a MATLAB or a Python function that takes $\frac{K+1}{2}$ non negative filter sampling values and produces K filter coefficients $h[n]$. Note that Matlab indexing starts at 1, so `H(1)` is $|H[0]|$ and `h(1)` is $h[0]$ etc. We leave up to you which of the two methods (or both?) you want to implement.

H3 No hint..

Q3 Instead of giving you an implementation, you can use the following two example input-output pairs to test your implementation. In Matlab notation, Python will be similar:

```
Hspec1=[1 1 0.4 0]
hspec1=[-0.0433   -0.0237    0.2956    0.5429    0.2956   -0.0237   -0.0433]
Hspec2=[0 0.5 1 0.5 0 0 0 0]
hspec2=[0.0027    0.0079   -0.0333   -0.1168   -0.1412   -0.0233    0.1707
0.2667    0.1707   -0.0233   -0.1412   -0.1168   -0.0333    0.0079    0.0027]
```

Q4 Use your function to design a band pass filter of length $K = 19$ that passes signals between $[0.2\pi, 0.4\pi]$ and removes other frequences[2]. Plot the frequency response of your filter. Apply your filter on $y[n]$, name the result $y_{19}[n]$ and its FFT $|Y_{19}[k]|$. Plot $|Y_{19}[k]|$ against k. Compare to $|Y[k]|$ in Question 1 and comment on your result.

H4 No hint

A4 You can check that

```
HSpec=[0 0 1 1 0 0 0 0 0 0]
```

should give

```
hspec=[0.0070  0.0489 0.0744  0.0283 -0.0780 -0.1639 -0.1461
-0.0164 0.1406  0.2105 0.1406 -0.0164 -0.1461 -0.1639 -0.0780
 0.0283 0.0744  0.0489 0.0070]
```

[2] Or at least tries to remove...

The frequency response plot of this filter can generated by the Matlab code

```
[H,W]=freqz(hspec, 1);
plot(W, abs(H)); xlim([0, pi]);
```

and in Python

```
import matplotlib.pyplot as plt
import scipy.signal as scs
(W,H)=scs.freqz(hspec,1)
plt.plot(W, np.abs(H))
plt.xlim(0,np.pi)
```

The result is shown in Figure 2. When you apply the filter on $x[n]$ and calculate its FFT $|Y_{19}[k]|$, you obtain the result shown in Figure 3. This filter is not ideal as it has a wide transition between its passband and stopband as the spectral resolution of this filter is $\Delta\Omega = \frac{2\pi}{19} \approx 0.33$.

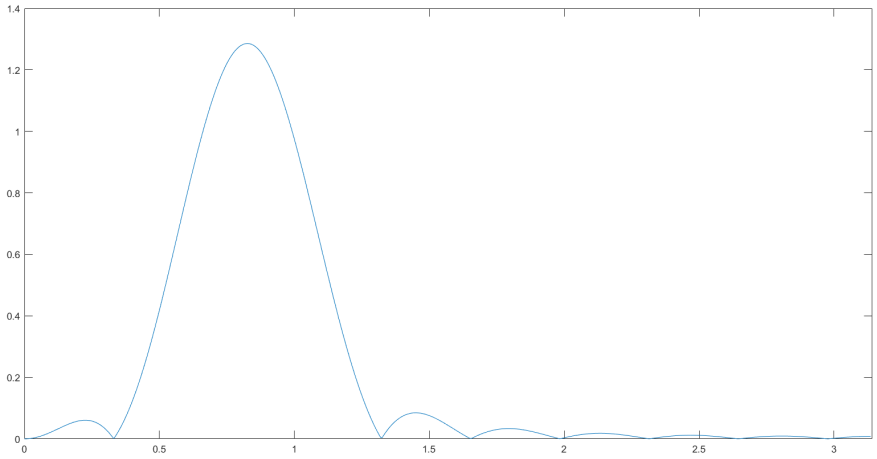


Figure 2: Frequency plot of $K = 19$ filter

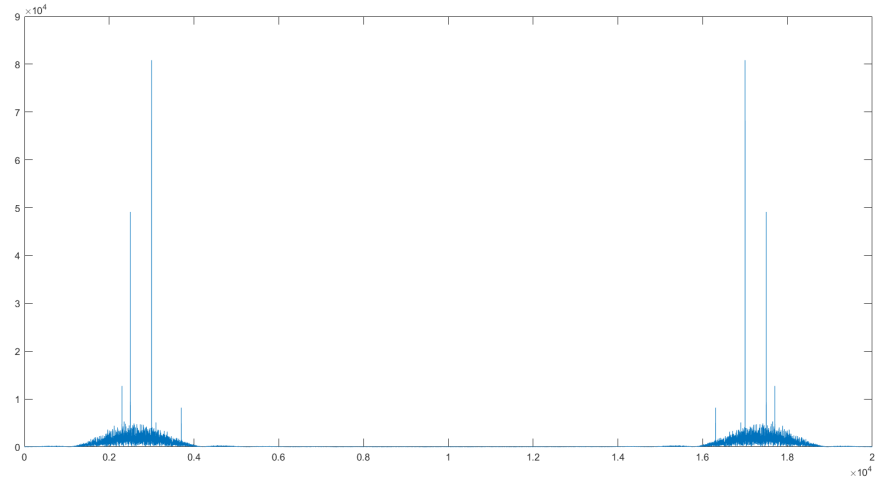


Figure 3: Plot of $|Y_{19}[k]|$

Q5 Repeat the previous question, but now use an extreme long filter of length $K = 1999$.
H5 Do you think we will specify the filter ;-)
A5 A linear frequency response plot of this filter is shown in Figure 2 and the effect of filtering in Figure 3. Although this filter is much better (spectral resolution $\Delta\Omega = \frac{2\pi}{1999} \approx 0.0031$ and a clear distinction between stopband and passband), the downside of its long length is the clear delay as seen in Figure 6.

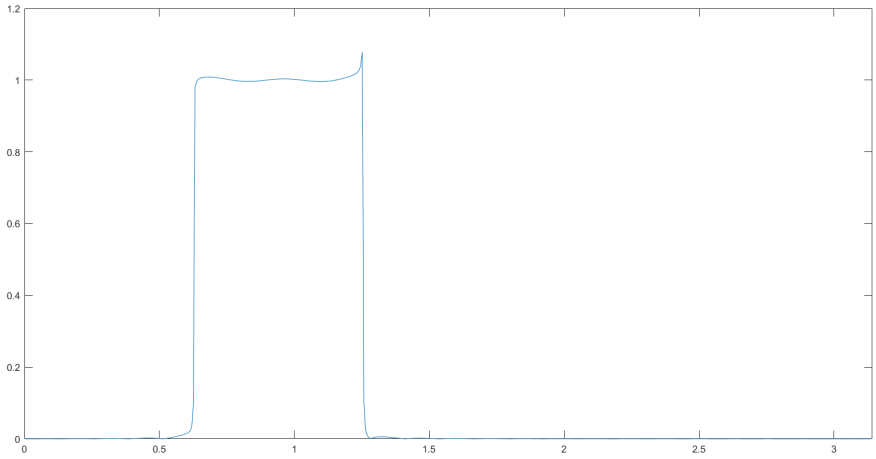


Figure 4: Frequency plot of $K = 1999$ filter

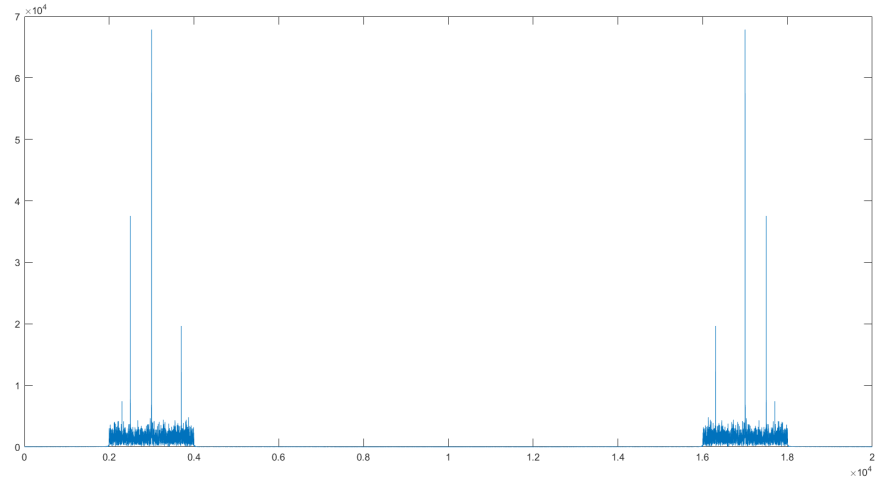


Figure 5: Plot of $|Y_{1999}[k]|$

Q6 What would a good choice of your filter length K that is good trade-off between being short enough and still having the right spectral resolution.
H6 Look at the indices of the peaks are located.
A6 The peaks are indices $k = 2300$ to $k = 3700$. The corresponding frequencies Ω range from $\frac{2\pi}{20000}2300$ to $\frac{2\pi}{20000}3700$ or from 0.23π to 0.37π. It seems that a spectral resolution of $\frac{\pi}{100}$ is a good choice as we can pinpoint frequencies 0.23π to 0.37π, resulting in a 200 length filter since $\Delta\Omega = \frac{2\pi}{200} = 0.01\pi$. However, this is an even filter length and if we allow for a little transition band, we might as well go for $K = 99$. In that case we need to find k_1 and k_2 such that

$$|H[k_1]| = 1, \cdots, |H[k_2]| = 1, \ |H[k]| = 0 \text{ for other values of } k,$$

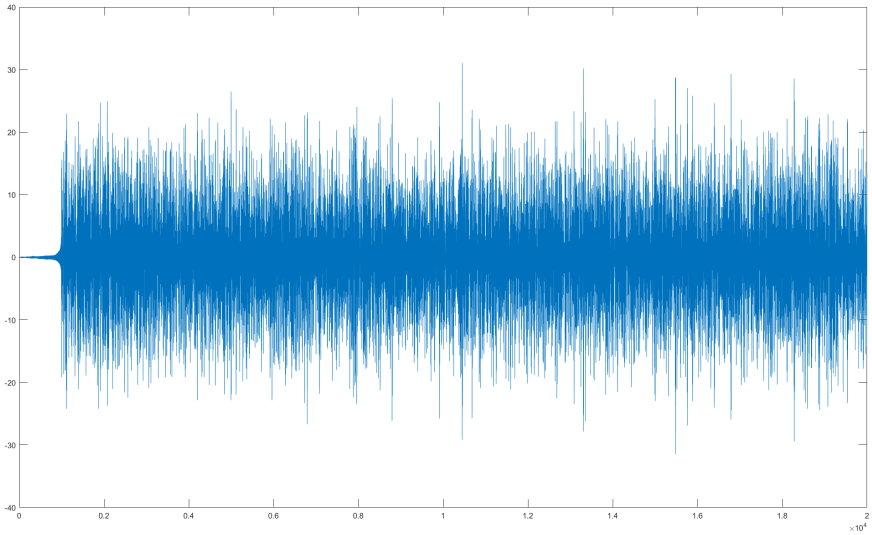


Figure 6: Plot of $y_{1999}[n]$, observe the delay caused the extreme length of the filter.

We have, with a filter length $K = 99$:

$$k_1 = \lfloor \frac{0.23\pi \cdot 99}{2\pi} \rfloor = 11, \;\; k_2 = \lceil \frac{0.37\pi \cdot 99}{2\pi} \rceil = 19.$$

These values match with frequencies $\Omega_1 = \frac{2\pi}{9} \approx 0.2222\pi$ and $\Omega_2 = \frac{38\pi}{99} \approx 0.3838\pi$. Note that we could also have used the specified interval $[0.2\pi, 0.4\pi]$ in which the signal should be present. It depends on the use case what you prefer and what is feasible. In the remainder we will just use the latter values.

Q7 For the remainder of this learning task, we use a $K = 99$ length filter and we pick $\Omega_1 = 0.2\pi$ and $\Omega_2 = 0.4\pi$. The previous method used the frequency sampling method. It is a flexible method as it enables you to design any filter you want. However, it can be cumbersome. For example, Matlab provide a filter design tool to quickly retrieve the four basic response types: lowpass, highpass, bandpass and bandstop. In Matlab, type `filterDesigner` and a design GUI shown in Figure 7 appears. Make sure you select the tab indicated by the red rectangle. If we want a $K = 99$ length filter, proceed as follows.

a) Select Reponse Type: in our case bandpass.

b) Select Design Method: FIR/Window (explained below).

c) Select order: 98, yields $K = 99$ length filter (Yes, it is confusing).

d) Select Window/Rectangular.

e) Select Units: Normalized.

f) Specify the two cut-off frequencies 0.2 and 0.4 (normalized in units of π).

g) Press Design Filter

You will see the magnitude response of the filter. Play with the icons above the magnitude response (green rectangle in Figure 7) to see for example the phase reponse (linear!). Through `File, Export` you export the filter to your Matlab workspace as `Num` and use it to filter $y[n]$. You do not have to use the visual tool, Matlab and Python provide commands to retrieve the same filter. In Matlab we have

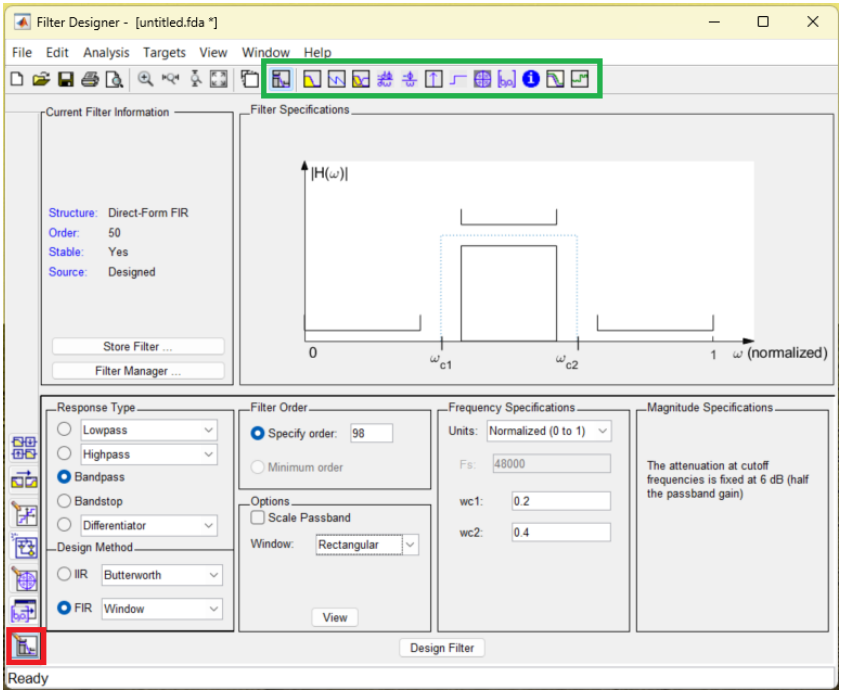


Figure 7: Screenshot Filter Designer

```
n=98;
hBP=fir1(n, [0.2, 0.4], "bandpass", rectwin(n+1), 'noscale');
```

and in Python

```
import numpy as np
import scipy.signal as scs
n=99
hBP=scs.firwin(n, np.array([0.2, 0.4]), window='boxcar',
scale=False, pass_zero='bandpass')
```

It is instructive to see the actual derivation of filter coefficients. It can be shown that the impulse response $h_{LP,id}[n]$ belonging to an ideal filter with cutoff frequency Ω_c is given by

$$h_{LP,ideal}[n] = \frac{\sin(\Omega_c n)}{\pi n}.$$

This filter cannot be realized in practice (why not?), so an alternative is to symmetrically truncate this filter outside $n \in \{-\frac{K-1}{2}, \frac{K-1}{2}\}$, making it a filter of finite length K. Truncation can be described as a multiplication with a window $w_K[n]$ which is 1 (0) inside (outside) $n \in \{-\frac{K-1}{2}, \frac{K-1}{2}\}$, respectively. Also, if we introduce a shift $\frac{K-1}{2}$, we obtain a practical causal low-pass filter

$$h_{LP,practical}[n] = \frac{\sin(\Omega_c(n - \frac{K-1}{2}))}{\pi(n - \frac{K-1}{2})} w_K[n - \frac{K-1}{2}].$$

In the case of $K = 99$, $\Omega_c = 0.1\pi$, and a rectangular window $w_{99}[n] = 1$ for $n \in \{-49, \cdots, +49\}$, 0 otherwise, this becomes

$$h_{LP,practical}[n] = \frac{\sin(0.1\pi(n-49))}{\pi(n-49)}, \; n \in \{-49, \cdots, 49\}, \; 0 \text{ otherwise}$$

We are not there yet. This might be a practical filter, but it is a **lowpass** filter. One can show that under some additional technical requirements the filter can be modulated in time, resulting in a shift in frequency. In formula,

$$h_{BP}[n] = 2\cos(\Omega_0 n)h_{LP}[n]$$

transforms the lowpass filter with cutoff frequency Ω_c to a bandpass filter on

$$[\Omega_0 - \Omega_c, \Omega_0 + \Omega_c] \cup [-\Omega_0 - \Omega_c, -\Omega_0 + \Omega_c].$$

Hence, our bandpass filter becomes

$$h_{BP,practical}[n] = 2\cos(0.3\pi(n-49))\frac{\sin(0.1\pi(n-49))}{\pi(n-49)},\ n \in \{-49, \cdots, 49\},\ 0 \text{ otherwise}$$

This is one of many approaches that can change the frequency response type of a filter, options include

- Lowpass to bandpass transformation, see above.
- Lowpass to highpass transformation, use (assuming a shifted filter)

$$h_{HP}[n] + h_{LP}[n] = \delta[n - \frac{K-1}{2}].$$

- Bandpass to bandstop transformation. How?
- Combine filters in series.
- Combine filters in parallel.

Verify that the values calculated with the bandpass filter exactly match with what the filter design tool or command line calculated for you. Note that you can also select other window functions that exhibit some desirable properties in the frequency domain. **The moral of the story is that the use of a design tool is ok, as long you can explain what the tool is doing.**
H7 Note that $\frac{\sin(\Omega_c n)}{\pi n}$ for $n = 0$ equals $\frac{\Omega_c}{\pi}$.
A7 The Matlab code is

```
K=99;
n=-(K-1)/2:(K-1)/2;
rangeNeg=-(K-1)/2:-1;
midpoint=0;
rangePos=1:(K-1)/2;
Omega_c=0.1*pi;
Omega_0=0.3*pi;
hBPNeg=2*cos(Omega_0*rangeNeg).*sin(Omega_c*rangeNeg)./(pi*rangeNeg);
hBP0=2*cos(Omega_0*0)*Omega_c/pi;
hBPPos=2*cos(Omega_0*rangePos).*sin(Omega_c*rangePos)./(pi*rangePos);
h=[hBPNeg hBP0 hBPPos];
```

and the Python code

```
import numpy as np
K=99
n=np.arange(-(K-1)/2,(K-1)/2+1)
rangeNeg=np.arange(-(K-1)/2,0)
midpoint=0;
rangePos=np.arange(1,(K-1)/2+1)
Omega_c=0.1*np.pi;
Omega_0=0.3*np.pi;
hBPNeg=2*np.cos(Omega_0*rangeNeg)*np.sin(Omega_c*rangeNeg)/(np.pi*rangeNeg)
hBP0=2*np.cos(Omega_0*0)*Omega_c/np.pi
hBPPos=2*np.cos(Omega_0*rangePos)*np.sin(Omega_c*rangePos)/(np.pi*rangePos)
h=np.hstack((hBPNeg, hBP0, hBPPos))
```

Let's compare the ten values $h[0]$ to $h[9]$ using Matlab, the Python output is similar.

```
Num(1:10) % output design tool
-0.0024    0.0024    0.0104    0.0106   -0.0000
-0.0111   -0.0114   -0.0028    0.0028   -0.0000
hBP(1:10) % output Matlab command
-0.0024    0.0024    0.0104    0.0106   -0.0000
-0.0111   -0.0114   -0.0028    0.0028   -0.0000
h(1:10) % constructed by hand
-0.0024    0.0024    0.0104    0.0106   -0.0000
-0.0111   -0.0114   -0.0028    0.0028   -0.0000
```

Q8 Let us describe how you can estimate the SNR. Here are some facts

- Signal and noise are assumed to be statistically independent, so power of received signal $y[n]$ is sum of the powers of $x[n]$ and $d[n]$.
- If a signal $s[n]$ has DFT $S[k]$, then $\frac{1}{N}|S[k]|^2$ is the spectrogram or the power spectral density of the signal $s[n]$. It measures the contribution of frequency index k to the total power of the signal $s[n]$.
- An estimation of the power of (any) real valued signal $s[n]$ between $[\Omega_1, \Omega_2]$ with DFT $S[k]$ is

$$\hat{\sigma}^2(\Omega_1, \Omega_2) = \frac{2}{N^2} \sum_{k=k_1}^{k_2} |S[k]|^2$$

  with $k_i = \frac{N}{2\pi}\Omega_i$. We assume that $0 \leq \Omega_1, \Omega_2 \leq \pi$, since $s[n]$ is real valued.
- The estimated SNR is given by estimated signal power/estimated noise power over the interval $[0, \pi]$.

In order to estimate the noise power, select a frequency interval with only noise and calculate its power contribution and extrapolate it to $[0, \pi]$. In general, if you select $[\Omega_1, \Omega_2]$ to be signal free, then

$$\hat{\sigma}^2 = \hat{\sigma}^2(\Omega_1, \Omega_2)\frac{\pi}{\Omega_2 - \Omega_1}$$

estimates the total noise power. Estimate the SNR. We will only present some high level Matlab code from which you can extrapolate the full Matlab or Python code.

H8 $x[n] = y[n] - d[n]$, so what about power then?

A8 Suppose $h[n]$ is the bandpass filter on $[0.2\pi, 0.4\pi]$, then we can derive the corresponding bandstop filter $hinv[n]$ in Matlab

```
K=99;
% see the previous question for the bandpass to bandstop conversion
hinv=-h;
hinv((K-1)/2+1)=1-h((K-1)/2+1);
% remove the signal
yinv=filter(hinv, 1, y);
Yinv=fft(yinv);
```

Write a function `spectralestimation` that takes the FFT and the frequencyband to estimate the power within the specified frequency band. We get

```
% frequency band
lb=0.5*pi;
ub=pi;
% total power
ypower=spectralestimation(Y, 0, pi);
% noise power estimation on band, extrapolated to complete interval
dpower=spectralestimation(Yinv, lb, ub)*(pi/(ub-lb));
% signal power=total power - noise power
SNR_p=(ypower-dpower)/dpower
```

The estimated SNR is 0.2354. We know the true SNR since we generated $y[n]$ ourselves via

$$y[n] = \cos(0.23\pi n) + 4\cos(0.25\pi n) + 7\cos(0.3\pi n) + 2\cos(0.37\pi n) + 12N(0,1).$$

It can be shown that the true SNR is (inspect the amplitudes and noise variance)

$$SNR = \frac{\frac{1}{2}(1^2 + 4^2 + 7^2 + 2^2)}{12^2} \approx 0.2451,$$

which shows that our estimate is pretty close.

Q9 You can easily improve the SNR. How? Can you predict what the SNR will be?
H9 Filter?
A9 If you apply the bandpass filter h to the signal, then the noise presence shrinks from $[0, \pi]$ to $[0.2\pi, 0.4\pi]$, reducing the presence by a factor of 5. Hence, the SNR will be roughly 1.25.